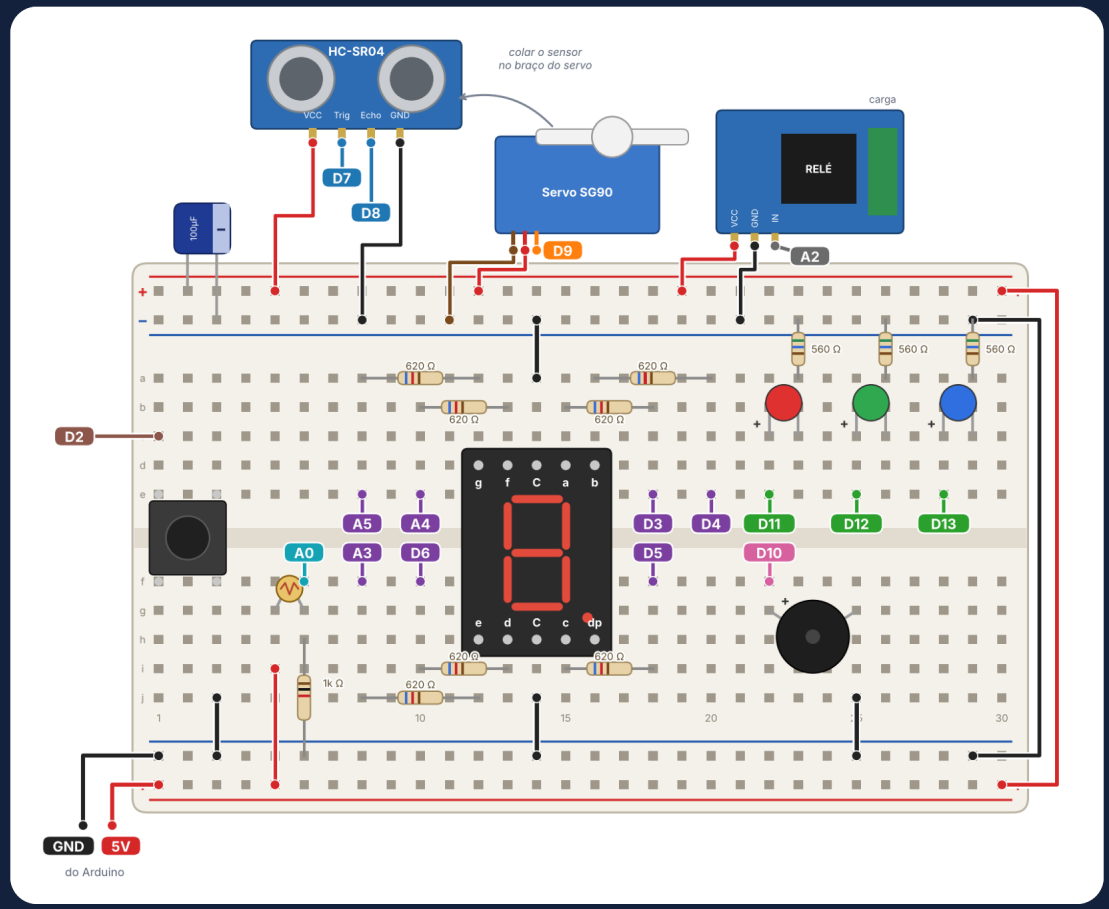
MINICURSO DE ROBÓTICA · PARTE 1

Arduino na prática, peça por peça

Montar e testar cada componente no lugar certo, até o corpo do Robô Sentinela ficar pronto.



Prof. Paulo César da Silva Junior · Engenharia de Computação

COMO VAMOS TRABALHAR

Uma peça de cada vez, sempre testando

1 · Entender
O que a peça faz e como ela funciona.

→

2 · Montar
No lugar definitivo da protoboard.

→

3 · Programar
Um código curto, só para ela.

→

4 · Testar
Funcionou? Só então a próxima.

As 10 etapas da Parte 1

0 Energia · 1 LED · 2 Três LEDs · 3 Botão · 4 Buzzer · 5 Sensor de luz · 6 Display · 7 Ultrassom · 8 Servo · 9 Relé

→ corpo do robô pronto

O QUE É UM ROBÔ

Perceber, decidir, agir — e repetir

Perceber

Sensores transformam o mundo em números.

Botão · sensor de luz · ultrassom

Decidir

Um programa escolhe o que fazer com esses números.

Arduino Uno

Agir

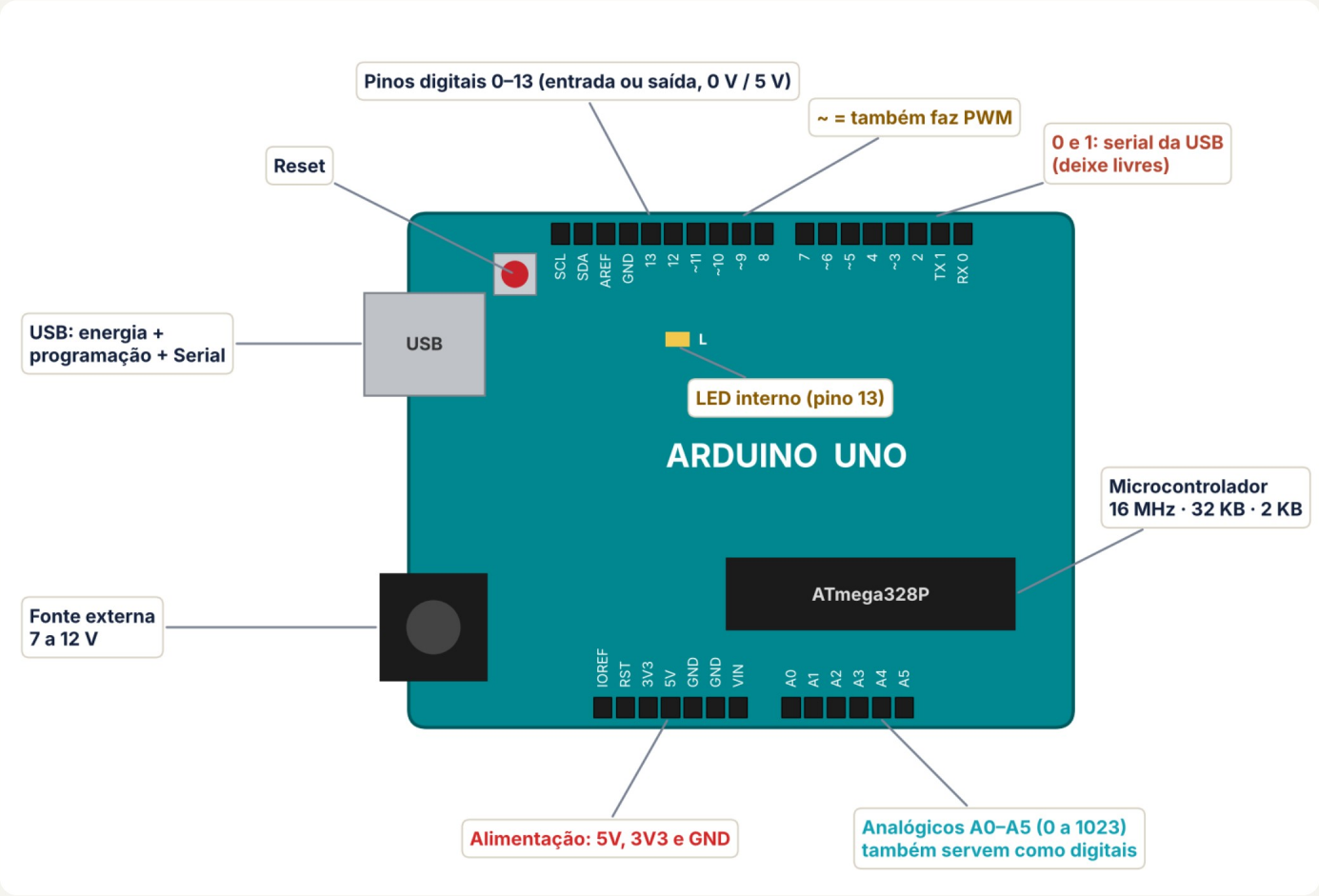
Atuadores mudam o mundo: luz, som, movimento.

LEDs · display · buzzer · servo · relé

Hoje montamos cada uma dessas peças. Na Parte 2, elas viram um robô sentinela que patrulha, percebe e reconhece gestos.

CONHECENDO · ARDUINO UNO

Um computador inteiro numa placa



16 MHz
velocidade do processador

32 KB
memória para o programa

2 KB
memória para as variáveis

Cada pino aguenta ~20 mA. Nunca ligue um motor direto nele.

CONHECENDO · ARDUINO IDE

Do código para a placa em 5 passos

1

Conectar

Cabo USB do Arduino ao computador. O LED **ON** acende.

2

Escolher a placa

Ferramentas › Placa › **Arduino Uno**

3

Escolher a porta

Ferramentas › Porta › **COM...** (Windows) ou **/dev/ttyUSB...**

4

Verificar e carregar

✓ compila e acha erros · → grava na placa

5

Ver o que acontece

Serial Monitor (texto) e **Serial Plotter** (gráfico), a 9600

⚠

Porta não aparece?

Placa compatível pede o driver **CH340**. Troque também o cabo USB.

CONHECENDO · PROGRAMAÇÃO

Todo programa tem duas partes

```
// 1. nomes para os pinos
const int LED = 11;

// 2. roda UMA vez ao ligar
void setup() {
  pinMode(LED, OUTPUT);
}

// 3. repete PARA SEMPRE
void loop() {
  digitalWrite(LED, HIGH);
  delay(500);
}
```

setup()

Prepara a placa: diz quais pinos são entrada ou saída e liga a comunicação.

loop()

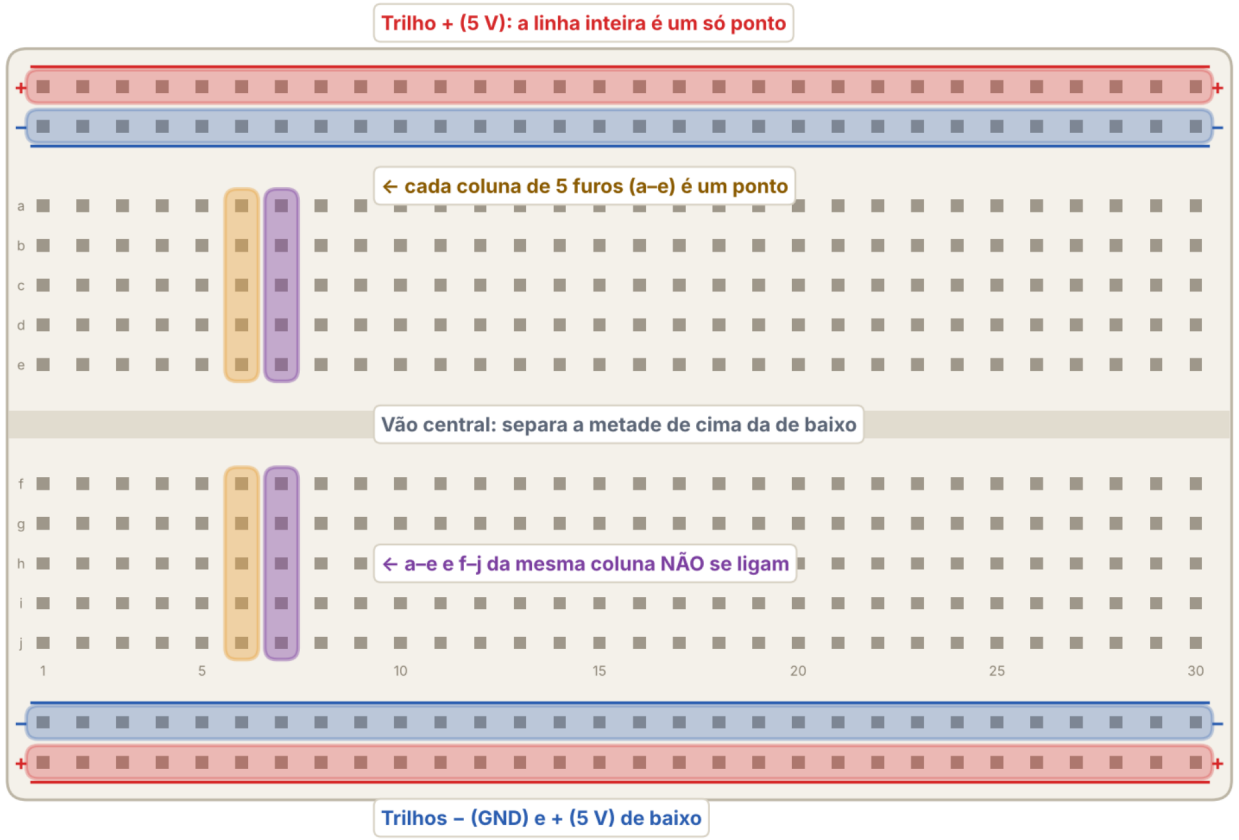
O trabalho de verdade: ler sensores, decidir, acionar. Repete milhares de vezes por segundo.

É C++

Ponto e vírgula no fim, chaves nos blocos, maiúsculas importam.

CONHECENDO · PROTOBOARD

Ligações sem solda: quais furos se conectam



Por baixo dos furos há **tiras de metal**. Tudo que está na mesma tira está ligado.

Colunas de 5 furos: onde as peças se encontram.

Trilhos: distribuem 5 V e GND para a placa toda.

Vão central: botões e o display ficam "a cavalo" sobre ele.

CONHECENDO · O KIT

Cada peça e quando ela entra

Peça	O que faz	Etapa
LEDs + resistores 560 Ω	Mostram estados com luz	1 e 2
Push button	Entrada do usuário	3
Buzzer	Bips, notas e sirene	4
LDR + resistor 1 kΩ	Mede a luz do ambiente	5
Display 7 seg. + 620 Ω	Mostra números e letras	6
HC-SR04	Mede distância com ultrassom	7
Servo SG90 + capacitor	Gira a "cabeça" (0° a 180°)	8
Módulo relé	Liga uma carga maior ("holofote")	9

Lei de Ohm e o código de cores

$$V = R \times I$$

O LED "consome" ~2 V. Sobram 3 V no resistor:

$$I = 3\ V / 560\ \Omega \approx 5\ mA$$

Acende bem e fica longe dos 20 mA do pino. Sem resistor, o LED (ou o pino) queima.

Valor	Faixas (cores)	Uso
560 Ω	verde · azul · marrom	LEDs
620 Ω	azul · vermelho · marrom	Display
1 kΩ	marrom · preto · vermelho	LDR

Leia a partir da faixa mais perto da ponta: **1º dígito, 2º dígito, nº de zeros**. A faixa dourada (5%) fica por último.

Resistor **não tem polaridade**: pode ir em qualquer sentido.

ANTES DE MONTAR

Quatro regras de ouro

1 **USB desligada para montar**

Mexeu em fio? Tire o cabo. Confira. Só então ligue de novo.

2 **Nunca 5 V direto no GND**

É um curto-circuito: a placa aquece e pode queimar.

3 **Respeite a polaridade**

LED, buzzer e capacitor têm lado + e lado –.

4 **Montou, não mexe mais**

Cada peça vai direto para o lugar final do robô.

Nos diagramas, as **etiquetas coloridas** (D11, A0…) dizem para qual pino do Arduino vai aquele fio. A peça nova de cada etapa aparece em destaque.

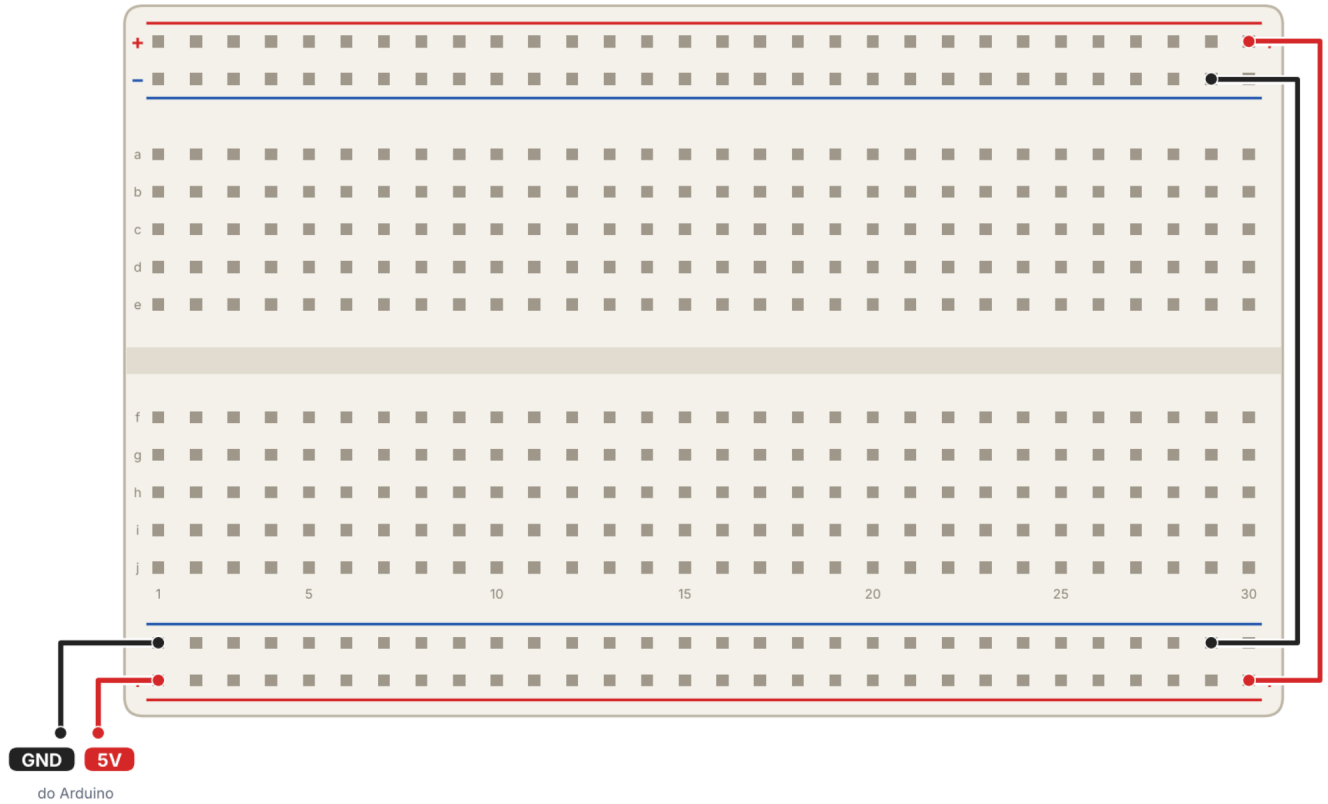
MÃO NA MASSA

Montagem passo a passo

Dez etapas. Cada uma com um diagrama para montar e um código para testar.

ETAPA 0 · MONTE

Energia nos trilhos



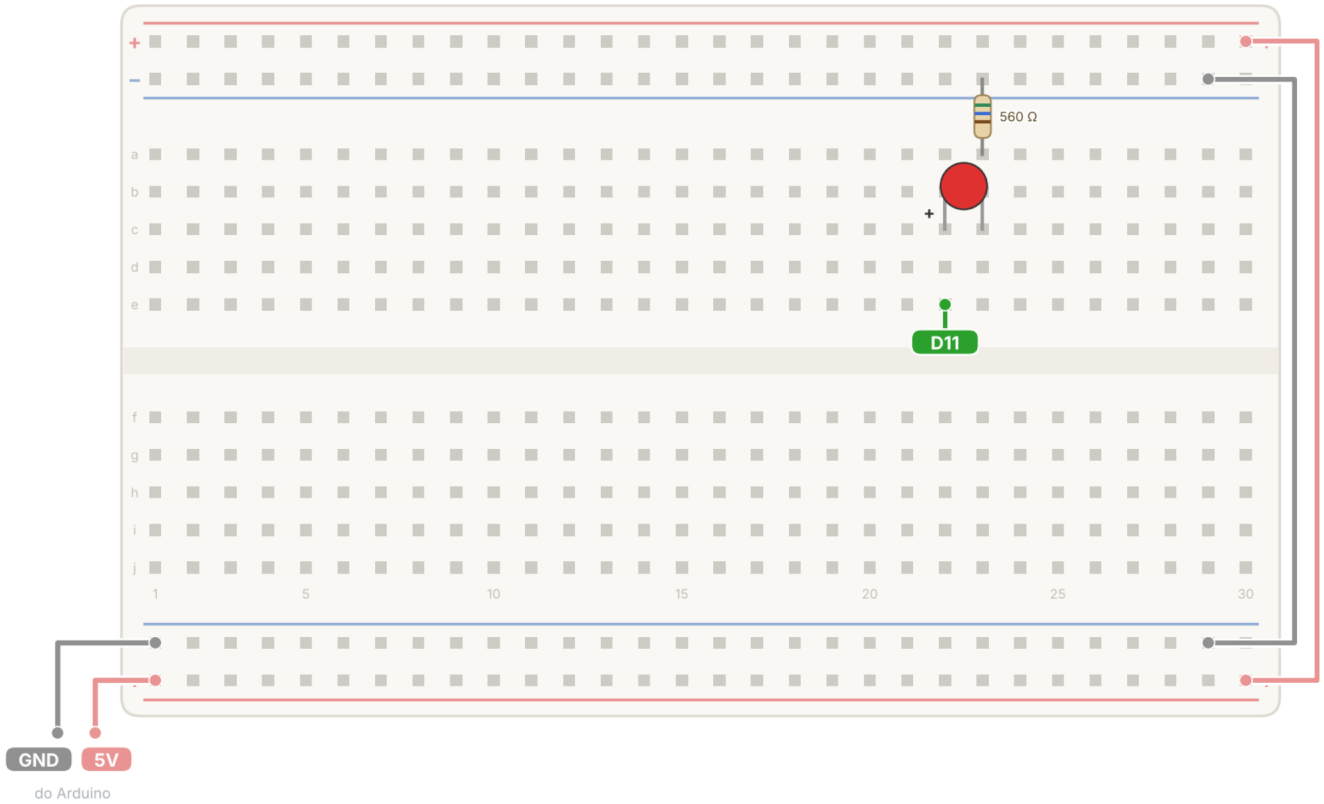
1. Fio vermelho: **5V** do Arduino → trilho **+** de baixo
2. Fio preto: **GND** do Arduino → trilho **–** de baixo
3. Na direita, dois jumpers ligam os trilhos de baixo aos de cima (+ com +, – com –)

Teste sem código

Ligue a USB: o LED **ON** acende e nada esquenta.

ETAPA 1 · MONTE

O primeiro LED



1. LED vermelho na linha c: perna **longa (+)** na coluna 22, curta na 23
2. Resistor **560 Ω** da coluna 23 ao trilho **–** de cima
3. Fio da coluna 22 ao pino **D11**

Caminho da corrente

D11 → LED → resistor → GND

ETAPA 1 · PROGRAME E TESTE · ETAPA1_LED

Fazer o LED piscar

```
const int LED_VERM = 11;

void setup() {
  pinMode(LED_VERM, OUTPUT);
}

void loop() {
  digitalWrite(LED_VERM, HIGH); // 5 V
  delay(500);
  digitalWrite(LED_VERM, LOW);  // 0 V
  delay(500);
}
```

Deve acontecer

O LED vermelho pisca: meio segundo aceso, meio apagado.

Por dentro

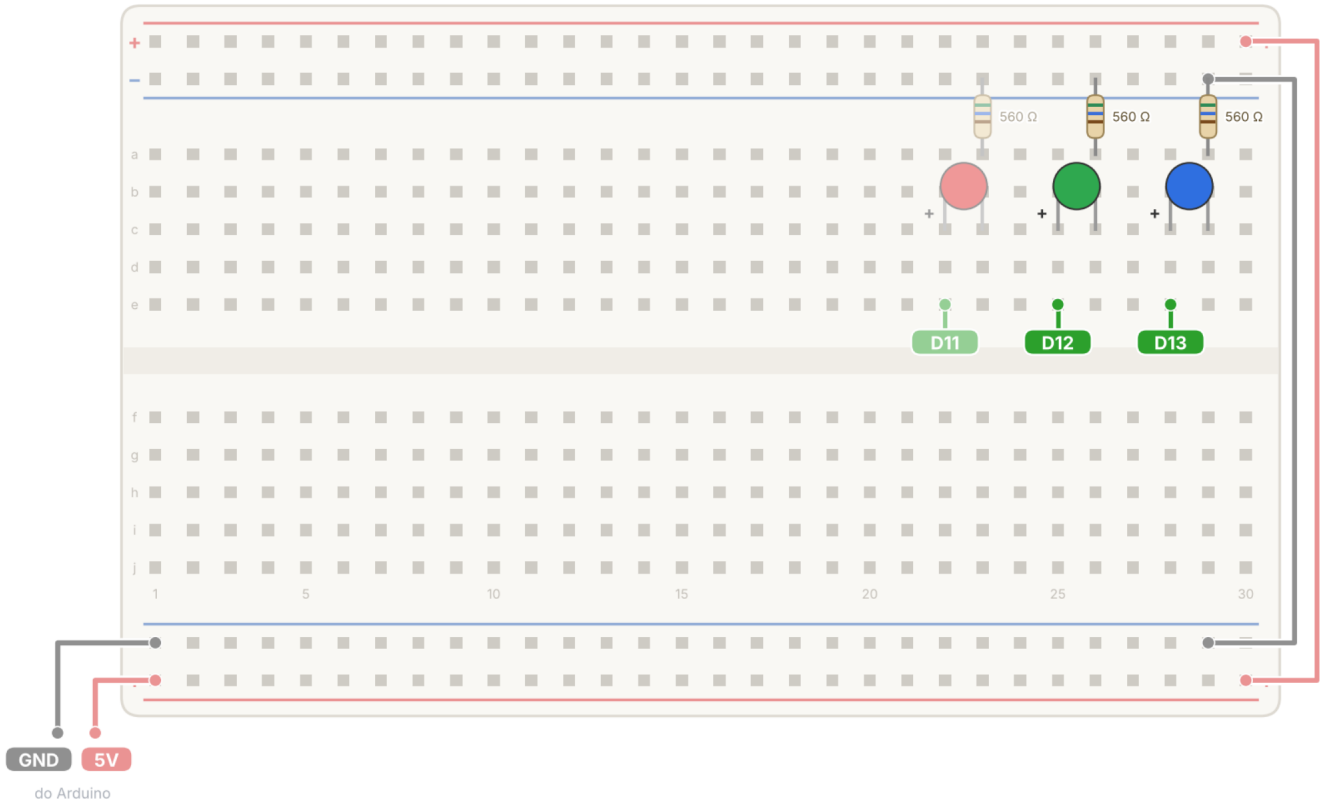
OUTPUT transforma o pino numa "chave": **HIGH** = 5 V, **LOW** = 0 V. **delay(ms)** espera sem fazer nada.

Desafio

Pisque "SOS" em Morse: curto 200 ms, longo 600 ms.

ETAPA 2 · MONTE

Mais dois LEDs



Mesmo padrão do LED vermelho:

LED	Colunas (+ / –)	Pino
Verde	25 / 26	D12
Azul	28 / 29	D13

Cada um com seu resistor de **560 Ω** até o trilho –.

ETAPA 2 · PROGRAME E TESTE · ETAPA2_LEDS

Luz correndo: vetor e for

```
// vermelho, verde, azul
const int LEDS[3] = {11, 12, 13};

void setup() {
  for (int i = 0; i < 3; i++)
    pinMode(LEDS[i], OUTPUT);
}

void loop() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(LEDS[i], HIGH);
    delay(300);
    digitalWrite(LEDS[i], LOW);
  }
}
```

Deve acontecer

Vermelho → verde → azul, em sequência, sem parar.

Por dentro

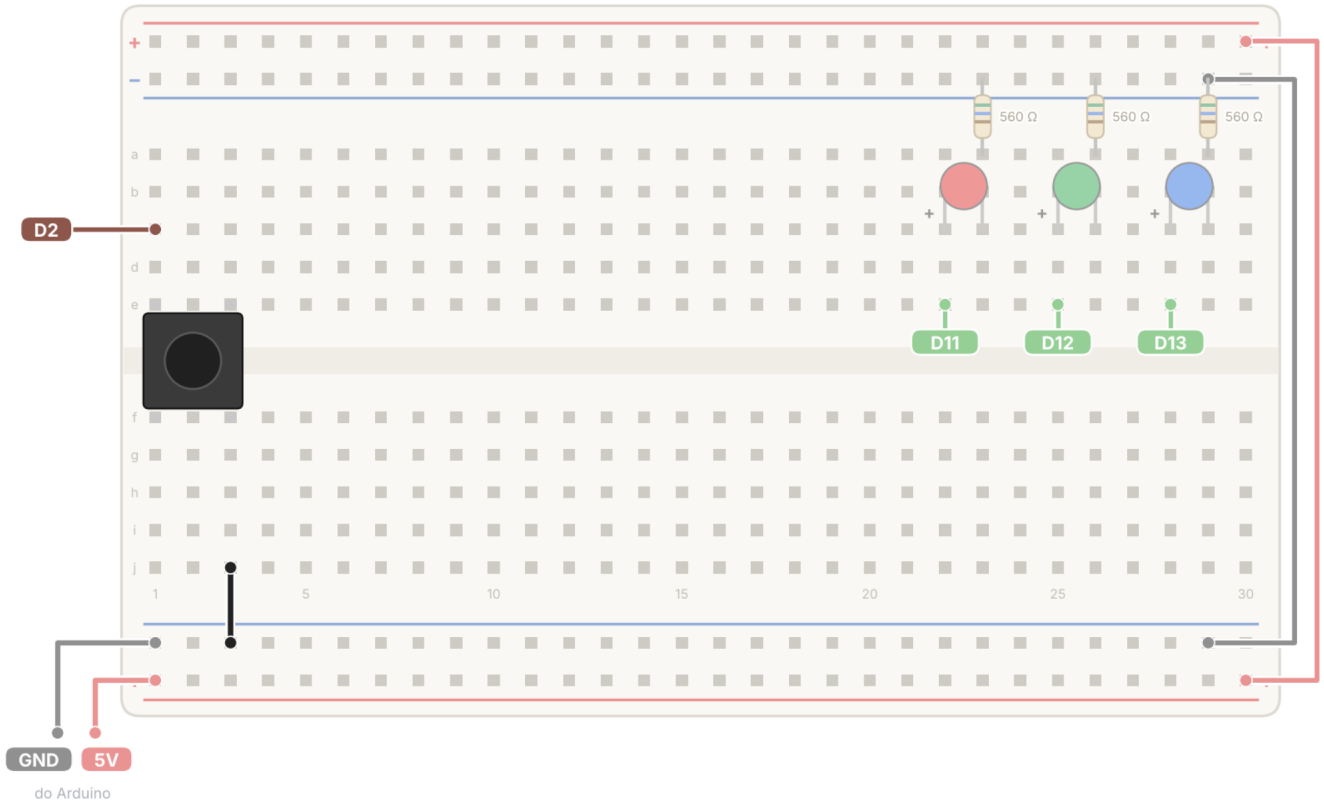
Um **vetor** guarda os 3 pinos; o **for** percorre o vetor. Vale para 3 ou para 30 LEDs.

Desafio

Faça a luz ir e voltar. Ou um semáforo: verde 3 s, azul 1 s, vermelho 3 s.

ETAPA 3 · MONTE

O botão



1. Botão "a cavalo" no vão central, pernas nas colunas 1 e 3
2. Coluna 1, metade de **cima** → pino **D2**
3. Coluna 3, metade de **baixo** → trilho **–**

Por que na diagonal?

Assim funciona com qualquer botão, não importa quais pernas ele liga por dentro.

ETAPA 3 · PROGRAME E TESTE · ETAPA3_BOTAO

Um aperto liga, outro desliga

```
const int BOTAO = 2, LED_VERDE = 12;
bool ledLigado = false;
int anterior = HIGH;  // solto = HIGH

void setup() {
  pinMode(BOTAO, INPUT_PULLUP);
  pinMode(LED_VERDE, OUTPUT);
}

void loop() {
  int leitura = digitalRead(BOTAO);
  if (anterior == HIGH && leitura == LOW) {
    ledLigado = !ledLigado;
    digitalWrite(LED_VERDE, ledLigado);
    delay(30);  // debounce
  }
  anterior = leitura;
}
```

Deve acontecer

Cada aperto liga ou desliga o LED verde. Segurar apertado não faz piscar.

Por dentro

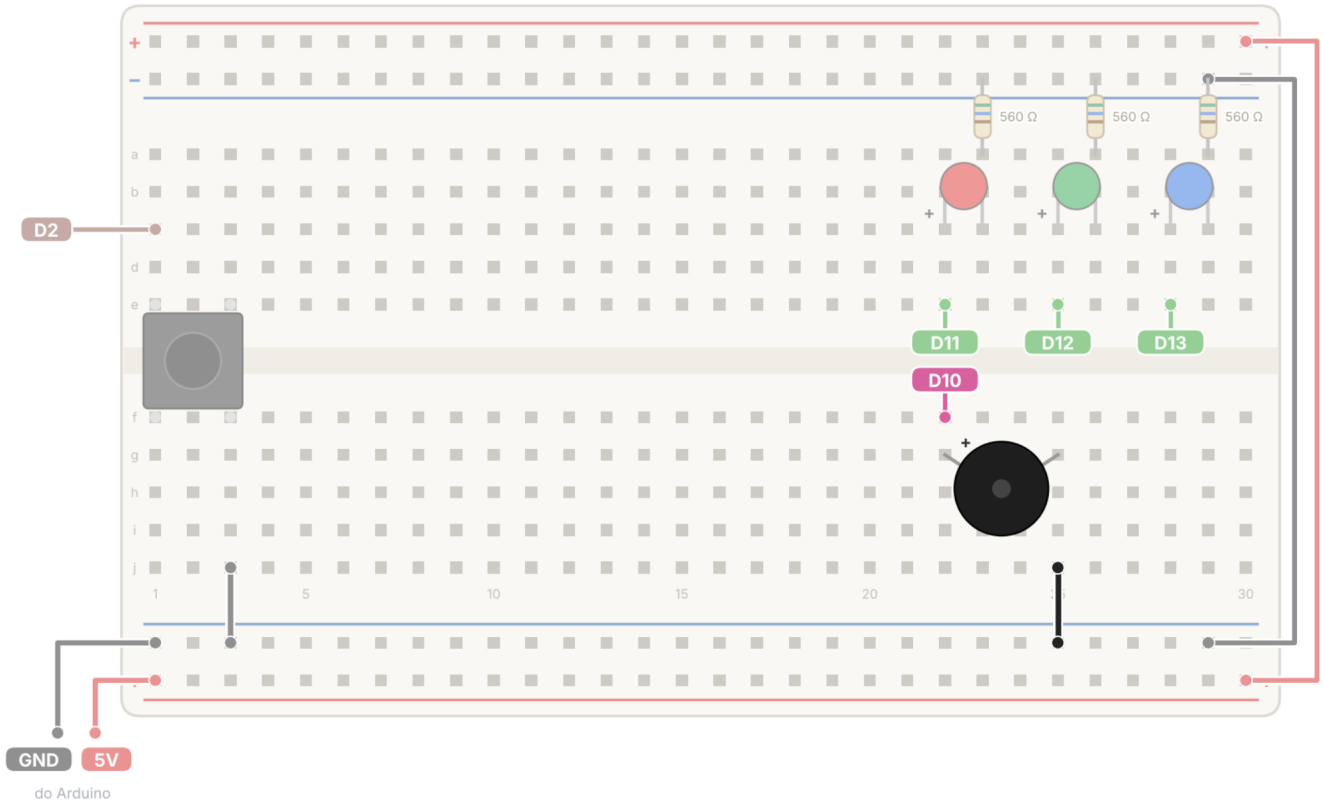
INPUT_PULLUP: solto lê HIGH, apertado lê LOW. Agimos só na **borda**: no instante em que passa de solto para apertado.

Desafio

Tire o delay(30) e aperte rápido. O que muda? Por quê?

ETAPA 4 · MONTE

O buzzer



1. Buzzer na metade de baixo: perna **+** na coluna 22, perna **–** na 25
2. Coluna 22 → pino **D10**
3. Coluna 25 → trilho **–** de baixo

Ativo ou passivo?

Passivo: toca notas. Ativo: só apita, numa nota fixa. O código tem um ajuste para cada um.

ETAPA 4 · PROGRAME E TESTE · ETAPA4_BUZZER

Botão toca a escala musical

```
const int BUZZER = 10, BOTAO = 2;
//          dó   ré   mi ... dó (Hz)
int notas[8] = {262, 294, 330, 349,
                392, 440, 494, 523};

void setup() {
  pinMode(BUZZER, OUTPUT);
  pinMode(BOTAO, INPUT_PULLUP);
}

void loop() {
  if (digitalRead(BOTAO) == LOW) {
    for (int i = 0; i < 8; i++) {
      tone(BUZZER, notas[i], 150);
      delay(170);
    }
  }
}
```

Deve acontecer

Ao apertar o botão: dó-ré-mi-fá-sol-lá-si-dó.

Por dentro

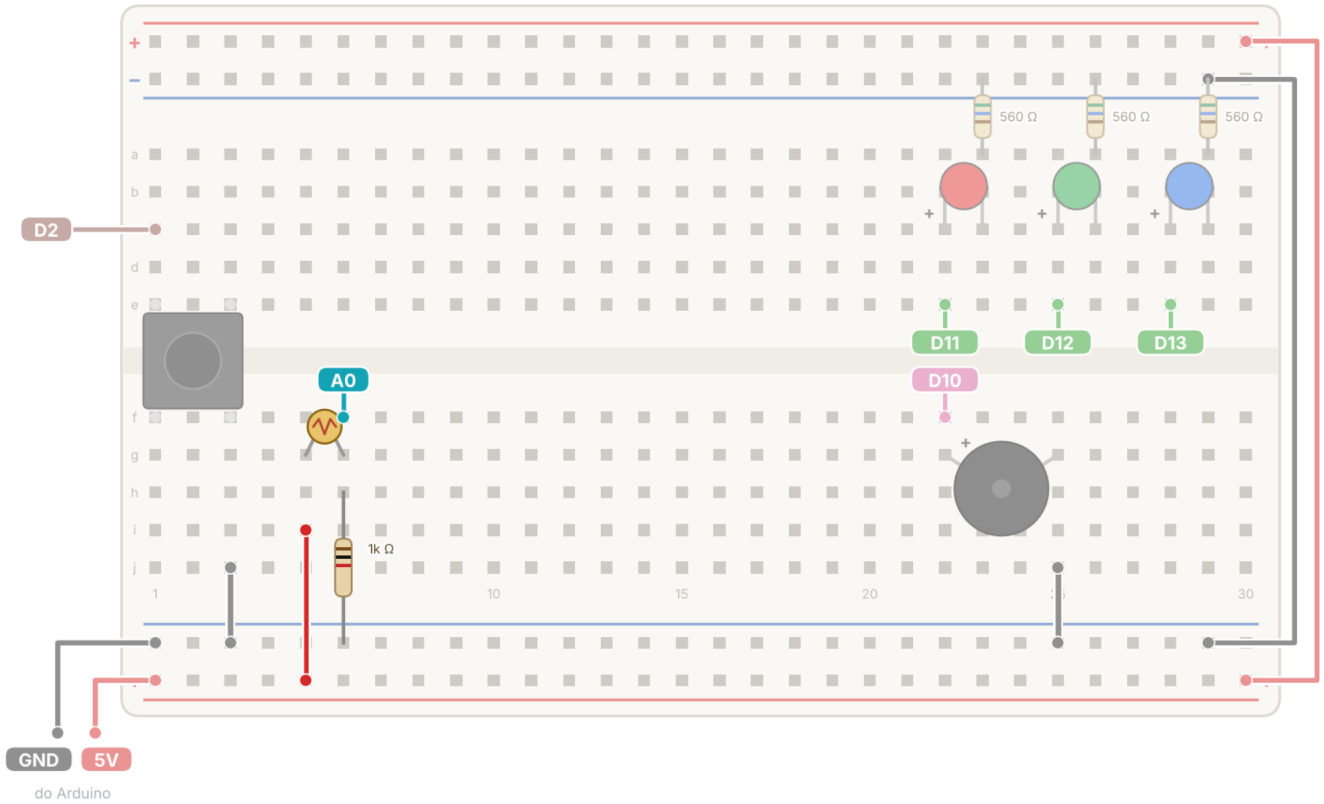
tone() liga e desliga o pino centenas de vezes por segundo. A **frequência** dessa onda quadrada é a nota: 440 Hz = lá.

Desafio

Faça uma sirene: alterne 900 Hz e 1500 Hz enquanto o botão estiver apertado.

ETAPA 5 · MONTE

O sensor de luz (LDR)



1. LDR nas colunas 5 e 6 (linha g)
2. Coluna 5 → trilho **+** (5 V)
3. Coluna 6 → pino **A0**
4. Resistor **1 kΩ** da coluna 6 ao trilho **–**

Divisor de tensão

Mais luz → LDR com menos resistência → mais tensão no A0.

ETAPA 5 · PROGRAME E TESTE · ETAPA5_LDR

Ler a luz e acender no escuro

```
const int LDR = A0, LED_AZUL = 13;
int LIMIAR = 150;  // ajuste depois de medir

void setup() {
  pinMode(LED_AZUL, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  int luz = analogRead(LDR);  // 0 a 1023
  Serial.print("luz:");
  Serial.println(luz);
  digitalWrite(LED_AZUL, luz < LIMIAR);
  delay(100);
}
```

Deve acontecer

No **Serial Plotter**, a curva cai ao cobrir o LDR. Abaixo do limiar, o LED azul acende.

Por dentro

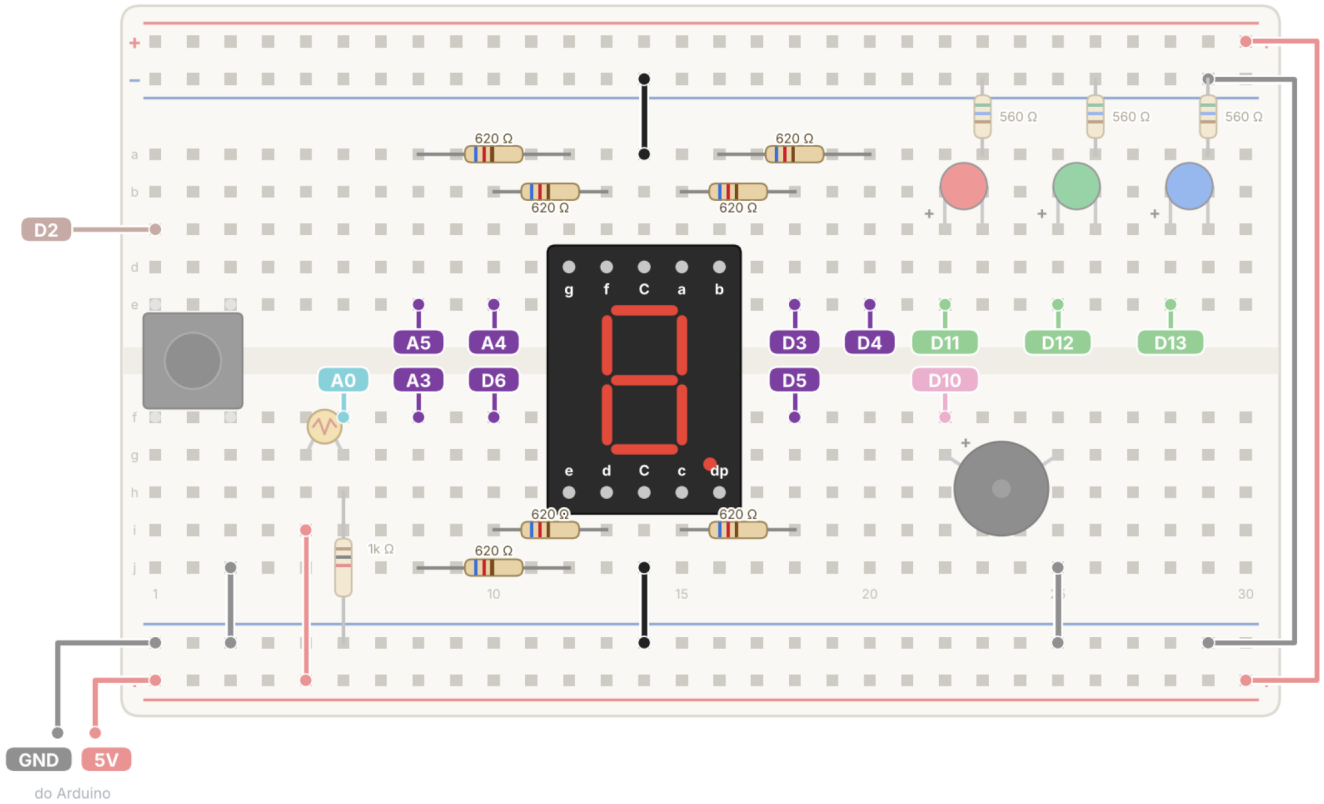
O conversor A/D tem **10 bits**: divide 0–5 V em 1024 degraus de ~4,9 mV.

Calibre (obrigatório)

Anote o valor no claro e coberto. Ponha o LIMIAR no meio. **Guarde esse número.**

ETAPA 6 · MONTE

O display de 7 segmentos



1. Display nas colunas 12–16, sobre o vão, ponto decimal embaixo à direita
2. Os dois pinos **C** (comum) → trilhos **–**
3. Cada segmento: resistor **620 Ω** até uma coluna livre; dela, o fio ao pino

a→D3	b→D4	c→D5	d→D6
e→A3	f→A4	g→A5	

Antes de ligar tudo, confirme os pinos do seu display (notas).

Contar de 0 a 9: cada número é um byte

```
//                    a  b  c  d  e  f  g
const int PIN_SEG[7] = {3, 4, 5, 6, A3, A4, A5};
const byte DIGITOS[10] = {0x3F, 0x06, 0x5B,
  0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F};

void mostrar(byte padrao) {
  for (int i = 0; i < 7; i++) {
    bool aceso = padrao & (1 << i);
    digitalWrite(PIN_SEG[i], aceso);
  }
}

void loop() {
  for (int n = 0; n ⩽ 9; n++) {
    mostrar(DIGITOS[n]);
    delay(600);
  }
}
```

Deve acontecer

0, 1, 2 ... 9 e recomeça.

Por dentro

Bit 0 = a ... bit 6 = g. O "2" acende a, b, d, e, g: **0101 1011** = **0x5B**.

Desafio

Escreva CAFE: C=0x39, A=0x77, F=0x71, E=0x79.

ETAPA 6B · INTEGRAÇÃO · ETAPA6B_CONTADOR

Juntando tudo: um contador

```
int contagem = 0;
int anterior = HIGH;

void loop() {
  int leitura = digitalRead(BOTAO);
  if (anterior == HIGH && leitura == LOW) {
    contagem = (contagem + 1) % 10;
    mostrar(DIGITOS[contagem]);
    digitalWrite(LED_VERM, contagem == 9);
    tone(BUZZER, 1200, 60);
    delay(30);
  }
  anterior = leitura;
}
```

Deve acontecer

Cada aperto soma 1 no display, com um bip. No 9 o LED vermelho acende; depois volta ao 0.

Repare

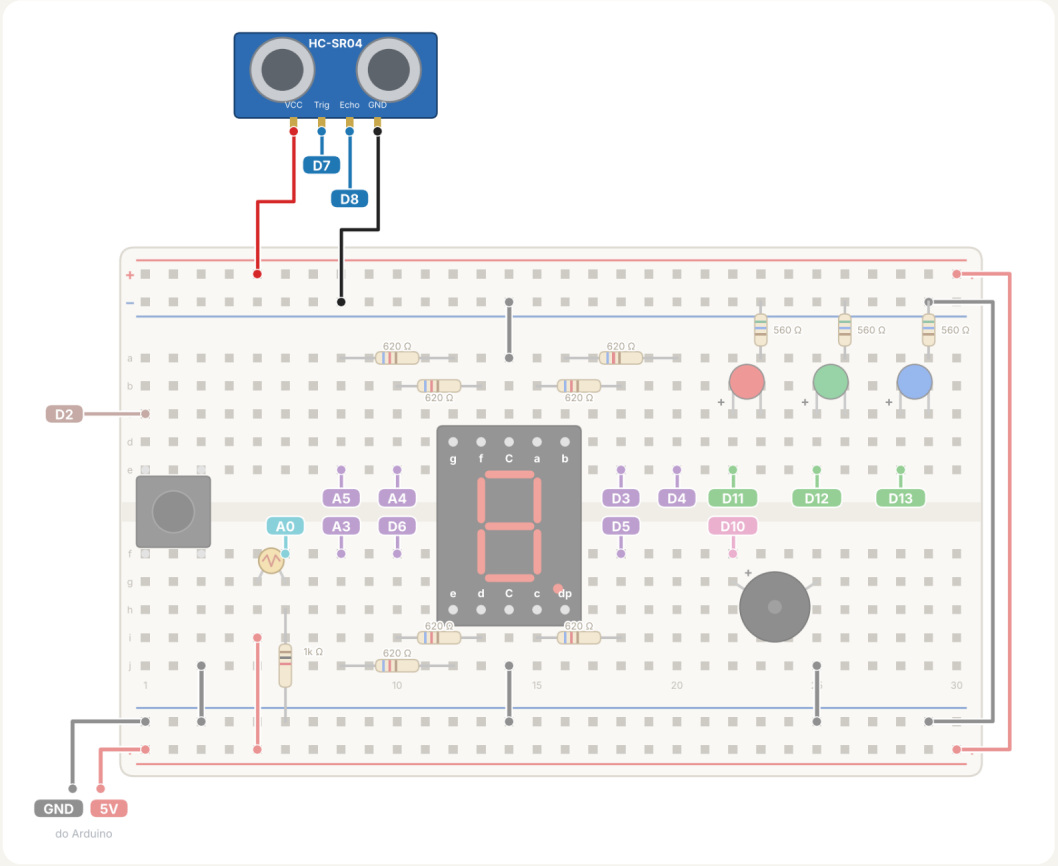
Nada novo foi montado. Três peças já testadas, combinadas pelo programa: isso é um **sistema**.

Desafio

Segurar o botão por 1 s zera a contagem (use millis()).

ETAPA 7 · MONTE

O sensor ultrassônico (HC-SR04)



Fica **fora** da protoboard: use 4 jumpers **macho-fêmea**.

Pino do sensor	Vai para
VCC	trilho + de cima (5 V)
Trig	pino D7
Echo	pino D8
GND	trilho – de cima

Como ele mede

Um "alto-falante" emite um apito de 40 kHz (inaudível); o "microfone" ouve o eco. O tempo do eco vira distância.

ETAPA 7 · PROGRAME E TESTE · ETAPA7_ULTRASSOM

Medir distância com o som

```
float lerDistanciaCm() {
  digitalWrite(PIN_TRIG, HIGH);
  delayMicroseconds(10);   // pulso de disparo
  digitalWrite(PIN_TRIG, LOW);
  unsigned long us =
    pulseIn(PIN_ECHO, HIGH, 12000UL);
  if (us == 0) return 200.0; // sem eco
  return us * 0.0343 / 2.0;
}

void loop() {
  float d = lerDistanciaCm();
  Serial.println(d);
  digitalWrite(LED_VERDE, d >= 40);
  digitalWrite(LED_AZUL, d >= 15 && d < 40);
  digitalWrite(LED_VERM, d < 15);
  delay(100);
}
```

Deve acontecer

Longe: verde. Médio: azul. Perto: vermelho. O display mostra a distância em dm.

Por dentro

`d = t × v / 2`

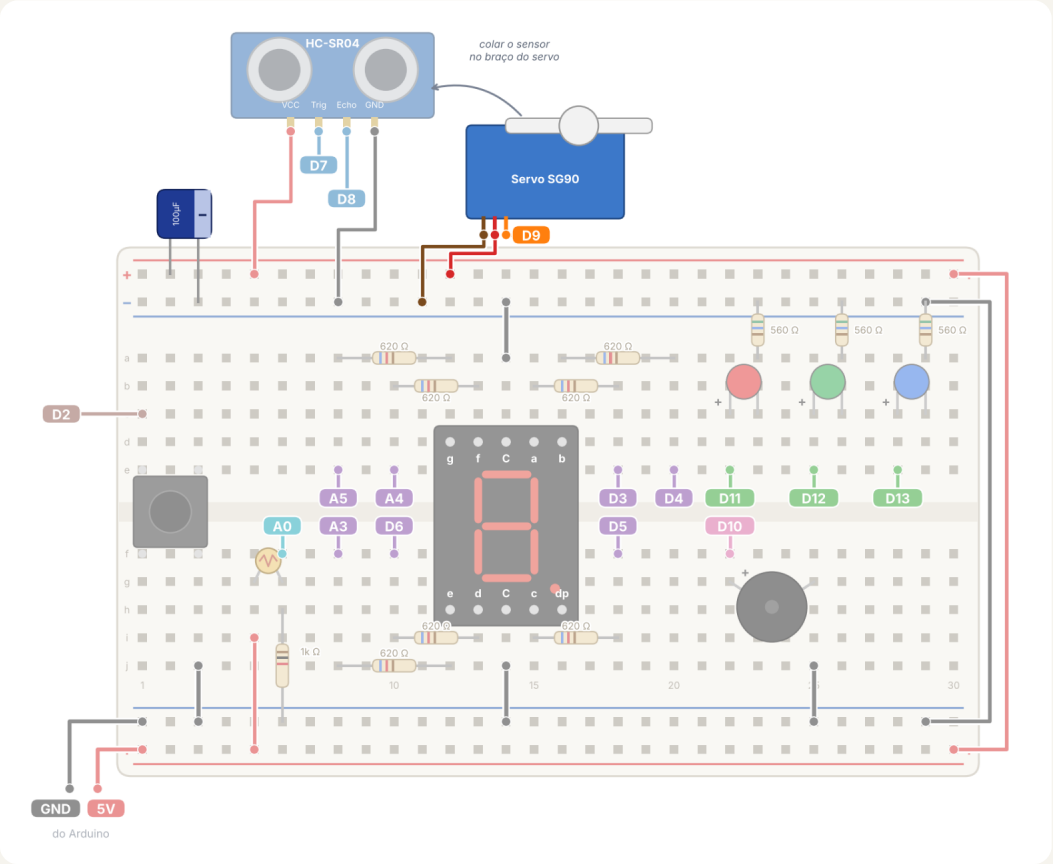
Som a 343 m/s = 0,0343 cm/µs. Divide por 2: ida e volta.

Desafio

Com uma régua: quanto o sensor erra a 10, 20 e 30 cm?

ETAPA 8 · MONTE

O servo: nasce a cabeça do robô



1. Servo: **marrom** → trilho –, **vermelho** → trilho +, **laranja** → **D9**
2. Capacitor **100 µF** entre os trilhos de cima, **faixa – no GND**
3. Prenda o servo numa base (caixinha ou papelão)
4. Cole o HC-SR04 no braço do servo, olhando para a frente

Por que o capacitor?

O motor puxa picos de corrente. Sem o capacitor, a tensão cai e o Arduino reinicia sozinho.

ETAPA 8 · PROGRAME E TESTE · ETAPA8_SERVO

A cabeça varre como um radar

```
#include <Servo.h>
Servo cabeca;

void medirEm(int angulo) {
  cabeca.write(angulo);
  delay(60);
  float d = lerDistanciaCm();
  digitalWrite(LED_VERM, d < 40);
  Serial.print(angulo);
  Serial.print(", ");
  Serial.println(d);
}

void loop() {
  for (int a = 20; a ⩽ 160; a += 4) medirEm(a);
  for (int a = 160; a ⩾ 20; a -= 4) medirEm(a);
}
```

Deve acontecer

A cabeça vai e volta de 20° a 160°. O LED vermelho acende quando ela "vê" algo perto.

Por dentro

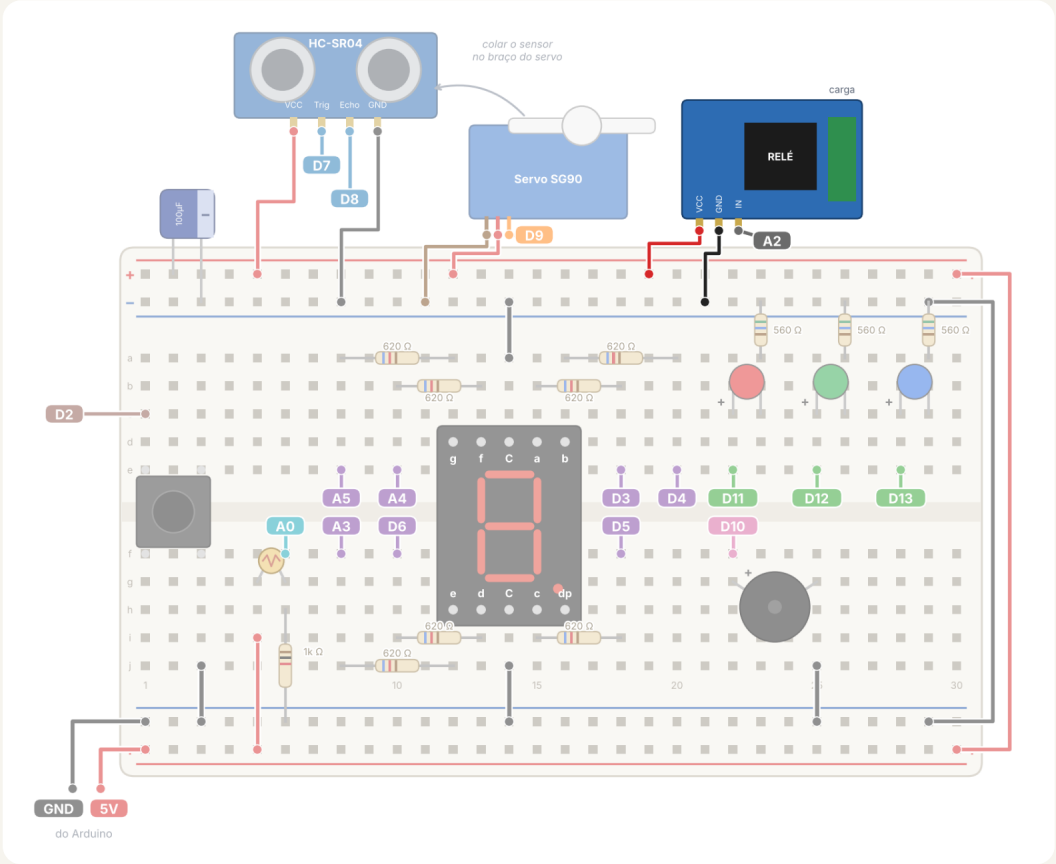
A cada 20 ms o servo recebe um pulso: **1 ms** = 0°, **1,5 ms** = 90°, **2 ms** = 180°. A biblioteca Servo gera isso para você.

Agora

Encaixe o braço do servo para que 90° aponte para a frente.

ETAPA 9 · MONTE

O relé: o ”holofote” do robô



Pino do módulo	Vai para
VCC	trilho + de cima (5 V)
GND	trilho – de cima
IN	pino A2

O que é um relé

Um interruptor acionado por um eletroímã. Com poucos miliampères do pino, liga uma carga isolada do Arduino.

Nada nos bornes verdes. Sem tensão de rede na bancada.

ETAPA 9 · PROGRAME E TESTE · ETAPA9_RELE

Escureceu, o holofote liga

```
const bool RELE_ATIVO_BAIXO = true;
int LIMIAR = 150;  // o da etapa 5

void rele(bool ligado) {
  digitalWrite(PIN_RELE,
    (ligado =≠ RELE_ATIVO_BAIXO) ? HIGH : LOW);
}

void loop() {
  bool escuro = analogRead(LDR) < LIMIAR;
  bool apertado = digitalRead(BOTAO) == LOW;
  rele(escuro || apertado);
  delay(200);
}
```

Deve acontecer

Cobrindo o LDR ou apertando o botão: "clique" e o LED do módulo acende.

Por dentro

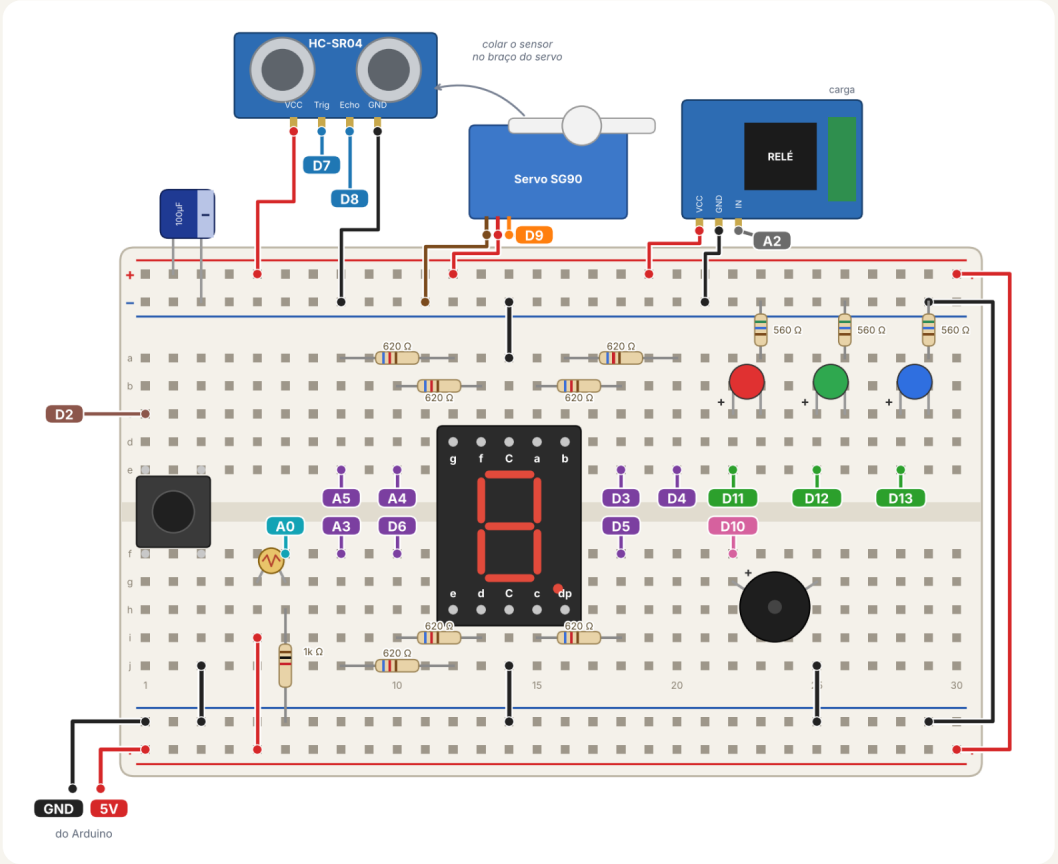
Muitos módulos ligam com **LOW** ("ativo baixo"). A função rele() esconde esse detalhe: o resto do código só pensa em ligar ou desligar.

Desafio

Na fronteira, o relé "tremula"? Use dois limiares: liga abaixo de 140, desliga acima de 170.

RESULTADO DA PARTE 1

O corpo do robô está pronto



Parte do robô	Peças
Olho	HC-SR04 (D7, D8)
Pescoço	Servo (D9)
Rosto	Display, 3 LEDs
Voz	Buzzer (D10)
Sentidos extras	LDR (A0), botão (D2)
Holofote	Relé (A2)

Cada peça foi testada sozinha. Falta o **cérebro**: um programa que use todas juntas.

TESTE GERAL · 01_TESTE_HARDWARE

Um código testa o robô inteiro

Carregue o **01_teste_hardware** e confira, nesta ordem:

1. LEDs vermelho, verde e azul
2. Display conta de 0 a 9
3. Buzzer: três notas
4. Servo vai a 0°, 180° e para em 90°
5. Relé dá um "clique"
6. Serial Plotter: distância, luz e botão

Anote para a Parte 2

- Display: cátodo ou ânodo comum?
- Buzzer: ativo ou passivo?
- Relé: liga com LOW ou HIGH?
- Valor do LDR no escuro (limiar)

Esses 4 ajustes ficam no topo de todos os códigos do robô.

FIM DA PARTE 1

O corpo está pronto. Agora, o cérebro.

Comportamento

Máquina de estados: patrulhar, prestar atenção, alarmar

Machine Learning

O robô aprende a reconhecer gestos com dados de vocês

Robô Sentinela

Tudo integrado no Arduino

Próximo: Parte 2 — Robô Sentinela com Machine Learning